

Advocall: The Rights Engine

Comprehensive Beginner-to-Expert Master Study Guide & Architectural Blueprint

Owner: Agastya (Rights Engine Lead) | Branch: `agastya` | Repo: Mohammed-Rayan07/advocall

Mission of this Guide

This document teaches you every detail of the component you personally built for Advocall. Even with zero prior coding experience, after studying this guide you will be able to navigate every file in `src/lib/rules/`, explain line-by-line what it does, why it exists, how it integrates with the rest of the system, and defend your work with confidence before hackathon judges.

Table of Contents

- SECTION 1: THE BIG PICTURE & SYSTEM ARCHITECTURE** — Problem definition, the deterministic rights philosophy, system data flow, and software concepts demystified.
- SECTION 2: FILE-BY-FILE & FUNCTION-BY-FUNCTION BLUEPRINT** — Detailed breakdown of every file in `src/lib/rules/` (Purpose, Who calls it, What it calls, Inputs/Outputs, and Failure modes).
- SECTION 3: STEP-BY-STEP MATHEMATICAL WALKTHROUGH** — Complete calculation tracing for the benchmark ₹4,500 UPI dispute.
- SECTION 4: COMPLETE HACKATHON JUDGE DEFENSE CHEAT SHEET** — 12 critical judge questions with concise verbal responses.
- SECTION 5: COMPREHENSIVE SELF-TEST & MASTERY VERIFICATION** — Quiz questions and answers to confirm total mastery.

SECTION 1: The Big Picture & System Architecture

1. What Problem Does Advocall Solve?

When an Indian consumer encounters an issue with a bank or online company (such as a failed UPI debit where their account is debited but the merchant is never credited, or an e-commerce platform that ignores a refund request), customer support agents routinely stall them with lines like: *"Please wait 7 to 10 working days, sir."*

Most consumers are unaware of their legal rights. Under Reserve Bank of India (RBI) regulations, banks must auto-reverse failed UPI debits within **T+1 day**. After that deadline, they legally owe the customer **₹100 per day** in statutory compensation. Advocall acts as an **autonomous AI advocate** that takes the consumer's case, calls customer support on their behalf, cites exact statutory rules to secure a formal complaint number, and prepares ready-to-file regulatory escalation packets if the company fails to deliver.

2. The Core Architectural Philosophy: Why the AI Never Computes Money

The Industry Flaw: LLM Hallucination in Law & Finance

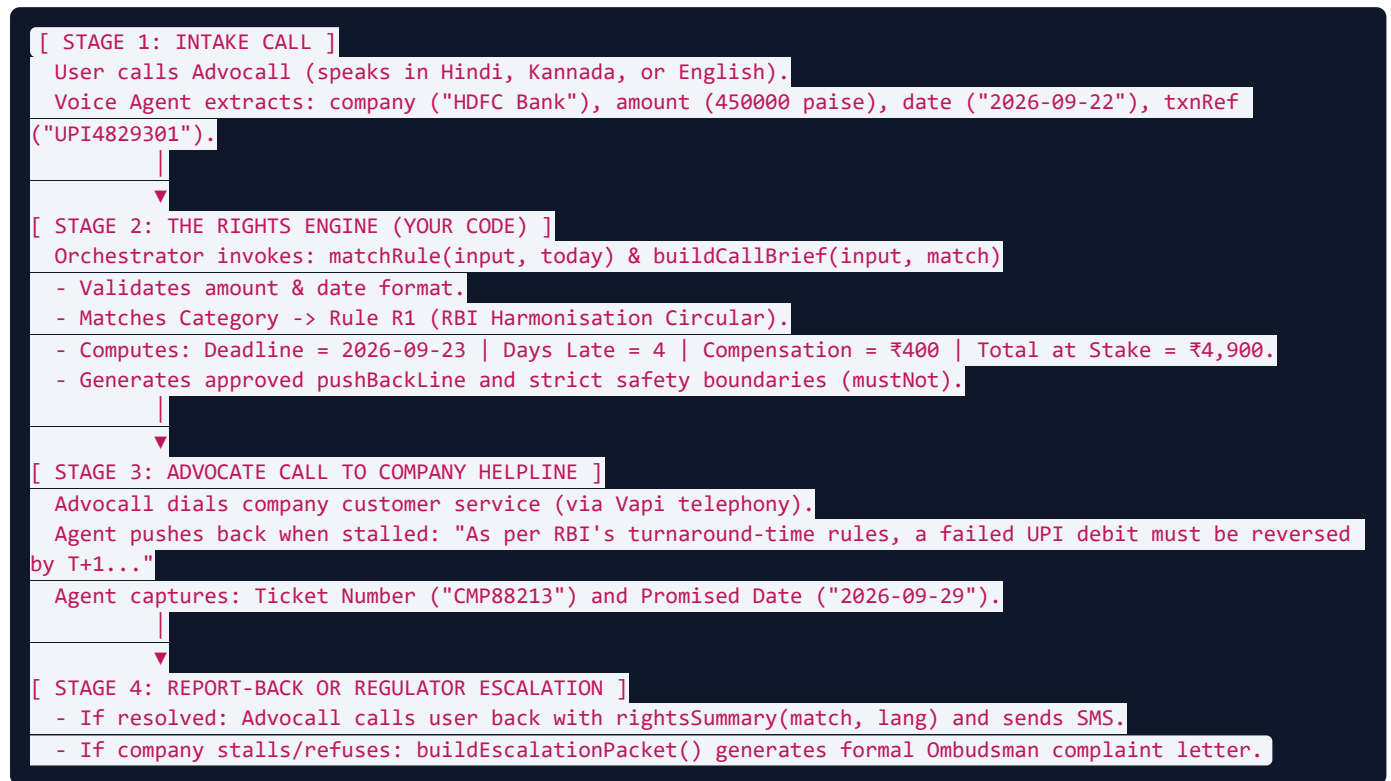
Existing tools like Pine AI or generic ChatGPT wrappers prompt a probabilistic Large Language Model (LLM) to "figure out the law and calculate compensation." This approach fails in consumer advocacy:

- LLMs are statistical text generators; they frequently produce mathematical errors.
- LLMs hallucinate non-existent regulations or mix up legal circular dates.
- Regulators (RBI Ombudsman, National Consumer Helpline) dismiss complaints that cite incorrect laws or inflated sums.

Advocall's Solution (Your Architecture): Strict separation of concerns.

- **The LLM (Voice Agent)** is strictly a conversational audio interface. It listens to the user, extracts basic facts (company name, amount, date), and speaks politely on the phone.
- **The Rights Engine (Agastya's Code)** is a 100% deterministic TypeScript calculation engine. It takes the facts and computes the exact deadline, the days overdue, and the accrued compensation down to the exact rupee. It provides the voice agent with *one approved, verbatim legal pushback sentence*. The LLM is programmatically forbidden from inventing laws or amounts.

3. End-to-End Data Pipeline



4. Demystifying Software Concepts for Beginners

Concept	What It Means in Plain English	How It Specifically Works in Advocall
TypeScript (TS)	A programming language that adds strict "type contracts" to JavaScript. If code expects an integer and receives text, it refuses to compile.	Guarantees <code>amountPaise</code> can never be a decimal or text string, catching bugs before code runs.
Next.js 16 (App Router)	A modern web framework built on React. It handles both frontend web pages and backend server routes in a unified system.	Runs the live dashboard (<code>http://localhost:3000</code>) and receives incoming telephony webhooks.
Pure Function	A coding function that behaves like pure math: given the exact same inputs, it ALWAYS produces the exact same output, with no outside side-effects.	<code>matchRule</code> and <code>daysBetween</code> are 100% pure. They never check the live system clock (<code>Date.now()</code>) so tests are always deterministic.
Paise Convention	Storing money as integer subunits (1 Rupee = 100 paise) instead of floating-point decimals.	Computers have rounding errors with decimals (e.g. <code>0.1 + 0.2 = 0.30000000000000004</code>). By storing ₹4,500 as <code>450000</code> integer paise, mathematical errors are physically impossible.

Vapi	A Voice AI platform that bridges phone networks (Twilio) with speech models (Speech-to-Text, LLMs, and Text-to-Speech).	Dials real phone numbers and executes the prompt scripts generated by our rights engine.
Barrel Export (index.ts)	A single file that collects functions from many individual files and exports them together in one clean gateway.	<code>src/lib/rules/index.ts</code> is the public gateway. Rayan's backend imports everything from <code>@lib/rules</code> without needing to know internal file names.

SECTION 2: File-by-File & Function-by-Function Blueprint

Every file in `src/lib/rules/` and `docs/RULEBOOK.md` belongs exclusively to you. Here is the exact architectural specification for every component.

1. `src/lib/rules/dates.ts` — Calendar Arithmetic Engine

Primary Purpose: Performs pure, timezone-safe calendar mathematics on `"YYYY-MM-DD"` strings. It eliminates timezone drift (e.g., UTC vs Indian Standard Time) and rejects invalid calendar dates.

Functions in `dates.ts` :

- `isValidYmd(ymd: string): boolean`
 - **What it does:** Validates whether an incoming string is a real calendar date in strict ISO format.
 - **Why we built it:** Prevents crashes or corrupt calculations caused by invalid dates like `"2026-02-30"`, non-leap year `"2026-02-29"`, or reversed formats like `"20-09-2026"`.
 - **How it works:** First uses regex `/^\d{4}-\d{2}-\d{2}$/`, then parses the numbers with `Date.UTC(y, m-1, d)` and checks if the resulting Date object matches the input year, month, and day.
 - **Called by:** `matchRule()` in `engine.ts` and `recordCommitment()` in voice orchestrator.
 - **Input / Output:** Input: `string` → Output: `boolean` (true/false).
 - **What breaks if removed:** Malformed date strings from voice calls crash the backend server.
- `addDays(ymd: string, days: number): string`
 - **What it does:** Adds an exact number of calendar days to a date string, returning a new `"YYYY-MM-DD"` string.
 - **Why we built it:** Calculates statutory deadlines (Incident + 1 day for UPI, Incident + 30 days for E-Commerce).
 - **How it works:** Converts date to UTC milliseconds using `Date.UTC()`, adds `days * 86,400,000 ms`, and converts back to ISO format.
 - **Called by:** `matchRule()`, `rightsTimeline()`.
 - **Input / Output:** Input: `(ymd: string, days: number)` → Output: `string`.
- `daysBetween(a: string, b: string): number`
 - **What it does:** Calculates the integer difference in whole days between date `a` and date `b` (positive if `b` is after `a`, negative if before).
 - **Why we built it:** Determines exactly how many days late an institution is beyond the statutory deadline.
 - **Called by:** `matchRule()`, `escalation.ts`, `timeline.ts`.
 - **Input / Output:** Input: `(a: string, b: string)` → Output: `number`.

2. `src/lib/rules/data.ts` & `docs/RULEBOOK.md` — Legal Rulebook Database

Primary Purpose: The data dictionary storing verified Indian legal statutes. If a law is not verified here, the AI is programmatically blocked from citing it.

The Rules Registry (`RULES` record):

- **Rule `R1` : Failed UPI Transfer (Verified: true)**
 - **Source:** RBI Circular RBI/2019-20/67 (DPSS.CO.PD No.629/02.01.014/2019-20), *Harmonisation of Turn Around Time (TAT) and customer compensation*, 20 Sep 2019.
 - **Statute:** Auto-reversal by beneficiary bank latest on **T + 1 day**. Statutory penalty of **₹100 per day** for delay beyond T+1.

- **Escalation Target:** RBI Integrated Ombudsman (CMS) at <https://cms.rbi.org.in>.
- **Rule R2 : E-commerce Refund Not Processed (Verified: true)**
 - **Source:** Consumer Protection (E-Commerce) Rules, 2020, Rule 4(5), Dept of Consumer Affairs.
 - **Statute:** Grievance officer must acknowledge complaint within **48 hours** and resolve within **one month (30 days)**. No daily monetary compensation.
 - **Escalation Target:** National Consumer Helpline (1915 / consumerhelpline.gov.in).
- **Rule R3 : Telecom / Broadband Billing (Verified: false)**
 - **Status:** Marked `verified: false`. The voice agent makes zero legal claims and asks only for a docket number.
- **FALLBACK : General Consumer Complaint (No Rule Matched)**
 - **Status:** `claimable: false`, `citeText: ""`. Asks strictly for a standard complaint number and date.

3. `src/lib/rules/engine.ts` — The Legal Decision Core (`matchRule`)

Primary Purpose: Takes intake case data and current date, validates parameters, maps regulatory jurisdiction, and calculates deadlines, delay penalties, and agent pushback scripts.

Function: `matchRule(input: CaseInput, today: string): RuleMatch`

- **Input:** `input` (Case details: amount in paise, category, incident date, company) and `today` (current date string).
- **Output (`RuleMatch` object):**
 - `ruleId` : `"R1" | "R2" | "R3" | "FALLBACK"`
 - `claimable` : `boolean` (true only if rule is verified and not FALLBACK)
 - `deadline` : ISO string or `null`
 - `daysLate` : integer (0 or greater)
 - `compensationPaise` : integer (₹100/day for R1, ₹0 for others)
 - `totalAtStakePaise` : `amountPaise + compensationPaise`
 - `explanation` : 1–2 plain-English sentences displaying formatted rupees (via `formatINR`)
 - `pushBackLine` : Exact verbal sentence the advocate voice agent speaks on the phone.
- **What breaks if removed:** The voice agent cannot determine applicable law, cannot counter customer care stalling, and cannot compute compensation.

4. `src/lib/rules/brief.ts` — Voice Agent Mission Briefing

Primary Purpose: Compiles operational guardrails, mission objectives, and hard security boundaries for the live voice agent before initiating a call.

Function: `buildCallBrief(c: CaseInput, match: RuleMatch): CallBrief`

- **goal** : What the agent must obtain on the call (e.g., *"Get complaint number, reversal date, acknowledgement of compensation..."*).
- **facts** : 2–3 sentences stating customer name, company, amount, incident date, and transaction reference.
- **citeLine** : Authorized legal sentence from `match.rule.citeText` if claimable; otherwise empty string `""`.
- **pushBackLine** : Pre-scripted pushback line from `match.pushBackLine`.
- **mustNot (Hard Safety Boundaries):**
 1. Never ask for, accept, or repeat OTP, PIN, CVV, passwords, or full card numbers.
 2. Never agree to a settlement or accept compromise on customer's behalf.
 3. Never cite any law or regulation other than the approved cite line.
 4. Never push back more than twice.
 5. Never be rude, threatening, or sarcastic.
- **stopWhen (Call Termination Criteria):** Stop when ticket number & date are recorded, or if company refuses twice, or if company demands OTP/KYC.

5. `src/lib/rules/escalation.ts` — Legal Escalation Packet Generator

Primary Purpose: Generates a formal, audit-ready legal complaint letter and structured facts evidence table for filing with the RBI Ombudsman or National Consumer Helpline.

Function: `buildEscalationPacket(c: Case, match: RuleMatch, commitment: Commitment | null, today: string): EscalationPacket`

- **11-Point Structured Facts Array:** Exactly ordered array of `[label, value]` pairs:
 1. `"Case ID"`
 2. `"Complainant"`
 3. `"Company"`
 4. `"Amount"` (formatted INR)
 5. `"Transaction reference"` (or `"Not provided"`)
 6. `"Incident date"` (formatted date)
 7. `"Rule"` (rule title)
 8. `"Deadline"` (formatted date or `"Not applicable"`)
 9. `"Compensation owed"` (formatted INR)
 10. `"Ticket number"` (ticket or `"Not provided"`)
 11. `"Company promised by"` (formatted date or `"Not provided"`)
- **Formal Letter Body:** Plain-text complaint letter formatted with Date, To, Subject, salutation, factual background, statutory circular references, call outcome, relief requested, and complainant signature. Cleanly omits any `undefined`, `null`, or `NaN`.

6. `src/lib/rules/timeline.ts` — Rights Milestone Engine

Primary Purpose: Generates a chronological lifecycle of milestones for the case, enabling visual status tracking on the dashboard.

Function: `rightsTimeline(input: CaseInput, match: RuleMatch, commitment: Commitment | null, today: string): RightsMilestone[]`

- Produces milestones: `Incident date` → `Statutory deadline` → `Company promised date` → `Today` → `Escalate if unresolved` (day after promised date or deadline).
- Assigns chronological status (`"past"` | `"today"` | `"future"`) by comparing dates against `today` using `daysBetween` .

7. `src/lib/rules/summary.ts` — Multilingual Consumer Summary

Primary Purpose: Translates regulatory math into 1–2 plain-language sentences in English, Hindi (Devanagari), and Kannada.

Function: `rightsSummary(match: RuleMatch, lang: "en" | "hi" | "kn"): string`

- **English:** "Under RBI rules, your failed UPI debit was due by 23 Sep 2026 (T+1). It is 4 days late, so ₹400 compensation is owed, making ₹4,900 total at stake."
- **Hindi:** "आरबीआई नियमों के अनुसार, आपके विफल यूपीआई का पैसा 23 Sep 2026 (T+1) तक वापस आना चाहिए था। यह 4 दिन देर हो चुका है, इसलिए ₹400 मुआवजा बनता है और कुल दावा ₹4,900 का है।"
- **Kannada:** "ಆರ್‌ಬಿಐ ನಿಯಮಗಳ ಪ್ರಕಾರ, ವಿಫಲವಾದ ಯುಪಿಐ ಹಣವು 23 Sep 2026 (T+1) ಒಳಗೆ ಮರಳಬೇಕಿತ್ತು..."

8. `src/lib/rules/check.ts` & `brief.test.ts` — Self-Verification Harnesses

Primary Purpose: Automated test scripts using Node's native `node:assert` module to mathematically verify R1, R2, FALLBACK, timelines, briefings, and summaries without external test runners.

SECTION 3: Step-by-Step Mathematical Walkthrough

To defend your engine before judges, you must be able to trace this exact case study effortlessly.

The Benchmark UPI Dispute

- **Complainant:** Ravi Kumar
- **Bank:** HDFC Bank
- **Category:** `upi_failed` (Failed UPI Transfer)
- **Amount:** ₹4,500 (represented in code as `450000` paise)
- **Incident Date:** `2026-09-22` (Tuesday)
- **Evaluation Date (Today):** `2026-09-27` (Sunday)

Step-by-Step Execution Trace:

Calculation Step	Engine Function & Formula	Result / Output
1. Input Validation	<code>Number.isInteger(450000) && 450000 > 0</code> <code>isValidYmd("2026-09-22")</code>	PASSED (No error thrown)
2. Jurisdiction Resolution	<code>RULE_FOR["upi_failed"]</code>	Rule R1 (RBI Harmonisation Circular)
3. Statutory Deadline (T+1)	<code>addDays("2026-09-22", 1)</code>	<code>2026-09-23</code> (Wednesday)
4. Overdue Duration	<code>Math.max(0, daysBetween("2026-09-23", "2026-09-27"))</code>	4 days late
5. Accrued Compensation	<code>4 days × 10,000 paise (₹100/day)</code>	40,000 paise (₹400)
6. Total Amount at Stake	<code>450,000 paise (principal) + 40,000 paise (comp)</code>	490,000 paise (₹4,900)

Verbatim Line Given to Voice Agent:

"As per RBI's turnaround-time rules, a failed UPI debit must be reversed by T+1, with ₹100 per day compensation after that. Could you please register a complaint and give me the complaint number and the reversal date?"

SECTION 4: Complete Hackathon Judge Defense Cheat Sheet

Judges will test whether you truly understand what you built or just used an AI wrapper. Here are concise, verbally punchy answers to the 12 most critical questions.

1. "Isn't this just Pine AI or Google Duplex?"

"No. Duplex books restaurant tables where there are zero legal consequences. Generic AI agents ask an LLM to guess refunds. Advocall is built around a deterministic Rights Engine: statutory deadlines, days overdue, and rupee penalties are computed by pure TypeScript code grounded in official RBI circulars. The AI never does the math or picks the law."

2. "Is it legal for an AI to call customer care on someone's behalf?"

"Yes. The user explicitly authorizes Advocall. In the very first sentence, the agent discloses: 'Hello, I am an AI assistant calling on behalf of [User], who is available to verify if needed.' It never impersonates the user. If identity verification or KYC is required, the agent offers to connect the customer directly."

3. "What if the AI hallucinates a non-existent law or quotes the wrong amount?"

"It is programmatically impossible. The voice agent is never allowed to generate legal arguments. It is passed one single approved string from our engine. If a category is unverified, our engine marks it unclaimable, stripping out all legal citations. Furthermore, the dashboard, SMS, and escalation letters pull directly from the engine's data, not the LLM's transcript."

4. "How do you handle OTPs, passwords, and banking privacy?"

"Our engine embeds hard safety invariants into the agent's briefing: never ask for, accept, or repeat OTPs, PINs, CVVs, passwords, or full card numbers. If a company representative asks for an OTP, the agent immediately stops and offers to hand over to the user."

5. "How does the system scale beyond UPI and E-commerce?"

"Our engine is completely data-driven. Adding an airline refund or insurance delay requires just adding a new entry in `src/lib/rules/data.ts` with verified statutory sources, defining the deadline logic, and adding unit tests. The calling pipeline, dashboard, and escalation generators require zero architectural changes."

6. "Why store money in paise instead of rupees?"

"To eliminate IEEE-754 floating-point rounding errors. In standard computing, decimals like $0.1 + 0.2$ produce `0.30000000000000004`. In financial disputes, rounding errors can invalidate regulatory filings. By using integer paise, ₹4,500 is stored as 450,000, guaranteeing exact arithmetic down to the paise."

7. "Why did you write custom UTC date functions instead of using standard Date or moment.js?"

"Local JavaScript `new Date('2026-09-22')` parses dates based on the server or browser's local timezone. A server running in UTC would evaluate a date differently from a user in IST, causing off-by-one day bugs. Our `dates.ts` functions use `Date.UTC` exclusively, ensuring deterministic, timezone-independent calendar math."

8. "What happens if the customer support representative refuses to cooperate or hangs up?"

"Advocall is programmed to push back politely at most twice. The voice agent cites the statutory regulation. If the representative still refuses, our system closes the call and immediately triggers `buildEscalationPacket()`, generating a formal complaint letter addressed to the RBI Ombudsman or National Consumer Helpline."

9. "What is your business model?"

"The first consumer case is completely free. Subsequent cases operate on a small success-fee model on statutory compensation recovered that the user would have otherwise abandoned. Additionally, we offer a B2B white-label resolution engine for fintechs and neo-banks to streamline dispute management."

10. "What does an advocate call cost to execute?"

"Approximately ₹15 to ₹40 for a 3-minute telephony call, covering real-time speech-to-text, LLM reasoning, text-to-speech synthesis, and Twilio phone trunking. This cost is negligible compared to the ₹1,000 to ₹10,000 transaction amounts and ₹100/day statutory compensation recovered."

11. "Where is case data stored? How do you comply with DPDP 2023?"

"In this hackathon prototype, case state is held strictly in volatile server memory—it completely purges on server reset, with zero external databases. For enterprise production, we implement end-to-end encryption at rest, strict 30-day retention policies, and explicit user consent matching India's Digital Personal Data Protection Act."

12. "What was your exact personal role on the team?"

"I am the Rights Engine Lead. I built the legal logic in `src/lib/rules/`, verified the RBI and E-Commerce statutory regulations, programmed the deterministic date and compensation math, wrote the escalation packet generator, created the multilingual summary engine, and validated everything with 100% passing automated test suites."

SECTION 5: Comprehensive Self-Test & Mastery Verification

Before stepping onto the stage, review these questions to confirm you have 100% command of your code.

Test Question 1: What happens if a user submits an amount of ₹0 or a negative number to `matchRule`?

Answer: `matchRule` immediately throws a descriptive JavaScript error (`amountPaise must be a positive integer`). It validates this before attempting any categorization or date math, protecting the system from garbage data.

Test Question 2: Why does Telecom (R3) have `verified` set to `false`? What does the engine do because of this?

Answer: Under our hackathon guidelines, we only cite statutes verified against primary government gazettes. Because TRAI complaint timelines were not officially verified during the sprint, R3 is marked `verified: false` . Consequently, `claimable` becomes `false` , `compensationPaise` is 0, and the engine strips out all legal citations, instructing the agent to ask neutrally for a standard docket number.

Test Question 3: How does the Rights Engine connect to the Voice Agent? Which function creates that bridge?

Answer: The bridge function is `buildCallBrief(caseInput, ruleMatch)` in `src/lib/rules/brief.ts` . It takes the output of `matchRule` and packages it into an operational brief containing the exact goal, case facts, approved citation line, pushback phrases, and 5 mandatory safety limits (`mustNot`).

Test Question 4: In `buildEscalationPacket`, what happens if the customer care call failed to capture a ticket number?

Answer: The function safely falls back to the exact string `"Not provided"` . It ensures the formal Ombudsman letter never contains missing values, `undefined` , `null` , or `NaN` .

Test Question 5: What are the three test commands you run to prove your code works?

Answer:

1. `npm run test:rules` (runs all 16 Vitest unit tests in `tests/rules.test.ts`).
2. `npm run typecheck` (runs `tsc --noEmit` to prove 0 TypeScript type errors).
3. `npx tsx src/lib/rules/check.ts` (runs the standalone assertion script verifying R1, R2, timelines, and multilingual summaries).